MLA02 – FUNDAMENTALS OF MACHINE LEARNING

LAB EXPERIMENT 4

Build an Artificial Neural Network by Implementing the Backpropagation Algorithm

Five Independent Problems with Aim, Procedure, Coding, Input, Output and Result

Name: S. MOHAN KUMAR

Register Number: 1925260331

Programs are written in Python 3 and use only the standard library.
Note: Problem 5 uses illustrative educational data only and is not a medical diagnostic tool.

Problem 1: Backpropagation Neural Network — XOR Logic

AIM

To implement a multilayer artificial neural network using backpropagation and learn the XOR logic function.

PROCEDURE

1. Import math and random to implement sigmoid activation and reproducible initial weights.
2. Prepare the XOR input-output training examples.
3. Initialize weights and biases for the input, hidden, and output layers.
4. Perform forward propagation to calculate the network output.
5. Calculate the output error and propagate it backward to update weights and biases.
6. Train for repeated epochs and test the network on a sample input.

PYTHON CODING

Run the complete code below using Python 3.

```
import math
import random

def sigmoid(value):
    # Clamp values to avoid overflow in exp().
    value = max(-60.0, min(60.0, value))
    return 1.0 / (1.0 + math.exp(-value))

def train_network(inputs, targets, hidden_count=4, epochs=5000, rate=0.5, seed=7):
    rng = random.Random(seed)
    input_count = len(inputs[0])
    # Small random weights and zero biases.
    w_hidden = [[rng.uniform(-0.5, 0.5) for _ in range(input_count)]
                for _ in range(hidden_count)]
    b_hidden = [0.0] * hidden_count
    w_output = [rng.uniform(-0.5, 0.5) for _ in range(hidden_count)]
    b_output = 0.0

    for _ in range(epochs):
        for x, target in zip(inputs, targets):
            # Forward propagation: input -> hidden layer.
            hidden = [
                sigmoid(sum(w * value for w, value in zip(weights, x)) + bias)
                for weights, bias in zip(w_hidden, b_hidden)
            ]
            # Hidden layer -> single sigmoid output.
            output = sigmoid(sum(w * value for w, value in zip(w_output, hidden))
                             + b_output)

            # Backpropagation: output-layer error signal.
            delta_output = (target - output) * output * (1.0 - output)
            old_w_output = w_output[:]
            for j in range(hidden_count):
                w_output[j] += rate * delta_output * hidden[j]
            b_output += rate * delta_output

            # Hidden-layer error signals use the old output weights.
            for j in range(hidden_count):
                delta_hidden = (delta_output * old_w_output[j]
                                * hidden[j] * (1.0 - hidden[j]))
                for k in range(input_count):
                    w_hidden[j][k] += rate * delta_hidden * x[k]
                b_hidden[j] += rate * delta_hidden

    def predict(x):
        hidden = [
            sigmoid(sum(w * value for w, value in zip(weights, x)) + bias)
            for weights, bias in zip(w_hidden, b_hidden)
        ]
        return sigmoid(sum(w * value for w, value in zip(w_output, hidden))
                       + b_output)

    return predict

def show_results(inputs, targets, names, sample, sample_name, hidden=4,
                 epochs=5000, rate=0.5, seed=7):
    model = train_network(inputs, targets, hidden, epochs, rate, seed)
    correct = 0
    print("Training predictions:")
    for x, target, name in zip(inputs, targets, names):
        probability = model(x)
        predicted = 1 if probability >= 0.5 else 0
        correct += int(predicted == target)
        print(f"{name}: target={target}, output={probability:.3f}, predicted={predicted}")
    print(f"Training accuracy: {100.0 * correct / len(targets):.1f}%")
    probability = model(sample)
    predicted = 1 if probability >= 0.5 else 0
    print(f"Test sample ({sample_name}): output={probability:.3f}, predicted class={predicted}")

inputs = [[0.0, 0.0], [0.0, 1.0], [1.0, 0.0], [1.0, 1.0]]
targets = [0, 1, 1, 0]
names = ["0 XOR 0", "0 XOR 1", "1 XOR 0", "1 XOR 1"]
sample = [1.0, 0.0]
sample_name = "1 XOR 0"
show_results(inputs, targets, names, sample, sample_name,
             hidden=4, epochs=7000, rate=0.5, seed=7)
```

INPUT

The training inputs, target labels, and new test sample are defined directly in the program.

OUTPUT

```
Training predictions:
0 XOR 0: target=0, output=0.015, predicted=0
0 XOR 1: target=1, output=0.979, predicted=1
1 XOR 0: target=1, output=0.984, predicted=1
1 XOR 1: target=0, output=0.027, predicted=0
Training accuracy: 100.0%
Test sample (1 XOR 0): output=0.984, predicted class=1
```

RESULT

The network learns the XOR examples using backpropagation and predicts class 1 for the test input [1, 0].

Problem 2: Backpropagation Neural Network — Student Result

AIM

To train a neural network using backpropagation to classify a student result from normalized attendance, assignment completion, and internal-test marks.

PROCEDURE

1. Represent attendance, assignment completion, and internal marks as numerical features between 0 and 1.
2. Label each training example as 1 for Pass and 0 for Fail.
3. Initialize a hidden layer and an output neuron with sigmoid activation.
4. Calculate predicted output using forward propagation.
5. Use the difference between target and predicted output to update weights by backpropagation.
6. Evaluate the trained network on the training records and a new student sample.

PYTHON CODING

Run the complete code below using Python 3.

```
import math
import random

def sigmoid(value):
    # Clamp values to avoid overflow in exp().
    value = max(-60.0, min(60.0, value))
    return 1.0 / (1.0 + math.exp(-value))

def train_network(inputs, targets, hidden_count=4, epochs=5000, rate=0.5, seed=7):
    rng = random.Random(seed)
    input_count = len(inputs[0])
    # Small random weights and zero biases.
    w_hidden = [[rng.uniform(-0.5, 0.5) for _ in range(input_count)]
                for _ in range(hidden_count)]
    b_hidden = [0.0] * hidden_count
    w_output = [rng.uniform(-0.5, 0.5) for _ in range(hidden_count)]
    b_output = 0.0

    for _ in range(epochs):
        for x, target in zip(inputs, targets):
            # Forward propagation: input -> hidden layer.
            hidden = [
                sigmoid(sum(w * value for w, value in zip(weights, x)) + bias)
                for weights, bias in zip(w_hidden, b_hidden)
            ]
            # Hidden layer -> single sigmoid output.
            output = sigmoid(sum(w * value for w, value in zip(w_output, hidden))
                             + b_output)

            # Backpropagation: output-layer error signal.
            delta_output = (target - output) * output * (1.0 - output)
            old_w_output = w_output[:]
            for j in range(hidden_count):
                w_output[j] += rate * delta_output * hidden[j]
            b_output += rate * delta_output

            # Hidden-layer error signals use the old output weights.
            for j in range(hidden_count):
                delta_hidden = (delta_output * old_w_output[j]
                                * hidden[j] * (1.0 - hidden[j]))
                for k in range(input_count):
                    w_hidden[j][k] += rate * delta_hidden * x[k]
                b_hidden[j] += rate * delta_hidden

    def predict(x):
        hidden = [
            sigmoid(sum(w * value for w, value in zip(weights, x)) + bias)
            for weights, bias in zip(w_hidden, b_hidden)
        ]
        return sigmoid(sum(w * value for w, value in zip(w_output, hidden))
                       + b_output)

    return predict

def show_results(inputs, targets, names, sample, sample_name, hidden=4,
                 epochs=5000, rate=0.5, seed=7):
    model = train_network(inputs, targets, hidden, epochs, rate, seed)
    correct = 0
    print("Training predictions:")
    for x, target, name in zip(inputs, targets, names):
        probability = model(x)
        predicted = 1 if probability >= 0.5 else 0
        correct += int(predicted == target)
        print(f"{name}: target={target}, output={probability:.3f}, predicted={predicted}")
    print(f"Training accuracy: {100.0 * correct / len(targets):.1f}%")
    probability = model(sample)
    predicted = 1 if probability >= 0.5 else 0
    print(f"Test sample ({sample_name}): output={probability:.3f}, predicted class={predicted}")

# Features: attendance fraction, assignment completion (0/1), internal marks fraction.
inputs = [
    [0.95, 1.0, 0.85], [0.85, 1.0, 0.75], [0.90, 1.0, 0.65],
    [0.75, 1.0, 0.80], [0.60, 1.0, 0.70], [0.45, 0.0, 0.30],
    [0.50, 0.0, 0.25], [0.35, 0.0, 0.40], [0.55, 1.0, 0.35],
    [0.30, 0.0, 0.20], [0.80, 0.0, 0.55], [0.65, 1.0, 0.60]
]
targets = [1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1]
names = [f"Student {i}" for i in range(1, len(inputs) + 1)]
sample = [0.82, 1.0, 0.72]
```

```
sample_name = "attendance=0.82, assignment=done, internal=0.72"
show_results(inputs, targets, names, sample, sample_name,
             hidden=5, epochs=6000, rate=0.4, seed=11)
```

INPUT

The training inputs, target labels, and new test sample are defined directly in the program.

OUTPUT

```
Training predictions:
Student 1: target=1, output=1.000, predicted=1
Student 2: target=1, output=1.000, predicted=1
Student 3: target=1, output=0.999, predicted=1
Student 4: target=1, output=1.000, predicted=1
Student 5: target=1, output=0.994, predicted=1
Student 6: target=0, output=0.004, predicted=0
Student 7: target=0, output=0.002, predicted=0
Student 8: target=0, output=0.009, predicted=0
Student 9: target=0, output=0.023, predicted=0
Student 10: target=0, output=0.000, predicted=0
Student 11: target=1, output=0.993, predicted=1
Student 12: target=1, output=0.976, predicted=1
Training accuracy: 100.0%
Test sample (attendance=0.82, assignment=done, internal=0.72): output=0.999, predicted class=1
```

RESULT

The trained neural network predicts class 1 (Pass) for the supplied student sample.

Problem 3: Backpropagation Neural Network — Customer Purchase

AIM

To use a backpropagation neural network to predict whether a customer will purchase a product from normalized visit frequency, promotion response, and previous purchase history.

PROCEDURE

1. Prepare numeric customer features scaled to the range 0 to 1.
2. Assign the target label 1 to purchase and 0 to no purchase.
3. Initialize network weights and biases using a fixed random seed.
4. Pass each input through the hidden and output layers.
5. Compute error gradients and update all weights and biases using backpropagation.
6. Print training predictions and classify a new customer sample.

PYTHON CODING

Run the complete code below using Python 3.

```
import math
import random

def sigmoid(value):
    # Clamp values to avoid overflow in exp().
    value = max(-60.0, min(60.0, value))
    return 1.0 / (1.0 + math.exp(-value))

def train_network(inputs, targets, hidden_count=4, epochs=5000, rate=0.5, seed=7):
    rng = random.Random(seed)
    input_count = len(inputs[0])
    # Small random weights and zero biases.
    w_hidden = [[rng.uniform(-0.5, 0.5) for _ in range(input_count)]
                for _ in range(hidden_count)]
    b_hidden = [0.0] * hidden_count
    w_output = [rng.uniform(-0.5, 0.5) for _ in range(hidden_count)]
    b_output = 0.0

    for _ in range(epochs):
        for x, target in zip(inputs, targets):
            # Forward propagation: input -> hidden layer.
            hidden = [
                sigmoid(sum(w * value for w, value in zip(weights, x)) + bias)
                for weights, bias in zip(w_hidden, b_hidden)
            ]
            # Hidden layer -> single sigmoid output.
            output = sigmoid(sum(w * value for w, value in zip(w_output, hidden))
                             + b_output)

            # Backpropagation: output-layer error signal.
            delta_output = (target - output) * output * (1.0 - output)
            old_w_output = w_output[:]
            for j in range(hidden_count):
                w_output[j] += rate * delta_output * hidden[j]
            b_output += rate * delta_output

            # Hidden-layer error signals use the old output weights.
            for j in range(hidden_count):
                delta_hidden = (delta_output * old_w_output[j]
                                * hidden[j] * (1.0 - hidden[j]))
                for k in range(input_count):
                    w_hidden[j][k] += rate * delta_hidden * x[k]
                b_hidden[j] += rate * delta_hidden

    def predict(x):
        hidden = [
            sigmoid(sum(w * value for w, value in zip(weights, x)) + bias)
            for weights, bias in zip(w_hidden, b_hidden)
        ]
        return sigmoid(sum(w * value for w, value in zip(w_output, hidden))
                       + b_output)

    return predict

def show_results(inputs, targets, names, sample, sample_name, hidden=4,
                 epochs=5000, rate=0.5, seed=7):
    model = train_network(inputs, targets, hidden, epochs, rate, seed)
    correct = 0
    print("Training predictions:")
    for x, target, name in zip(inputs, targets, names):
        probability = model(x)
        predicted = 1 if probability >= 0.5 else 0
        correct += int(predicted == target)
        print(f"{name}: target={target}, output={probability:.3f}, predicted={predicted}")
    print(f"Training accuracy: {100.0 * correct / len(targets):.1f}%")
    probability = model(sample)
    predicted = 1 if probability >= 0.5 else 0
    print(f"Test sample ({sample_name}): output={probability:.3f}, predicted class={predicted}")

# Features: visit frequency, promotion response, previous purchase (0/1).
inputs = [
    [0.90, 1.0, 1.0], [0.80, 1.0, 1.0], [0.70, 1.0, 0.0],
    [0.85, 0.0, 1.0], [0.60, 1.0, 1.0], [0.20, 0.0, 0.0],
    [0.30, 0.0, 0.0], [0.40, 0.0, 0.0], [0.25, 1.0, 0.0],
    [0.50, 0.0, 1.0], [0.65, 1.0, 1.0], [0.15, 0.0, 1.0]
]
targets = [1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 0]
names = [f"Customer {i}" for i in range(1, len(inputs) + 1)]
sample = [0.75, 1.0, 0.0]
```

```
sample_name = "frequent visits, responds to promotion, no previous purchase"
show_results(inputs, targets, names, sample, sample_name,
             hidden=5, epochs=6000, rate=0.4, seed=13)
```

INPUT

The training inputs, target labels, and new test sample are defined directly in the program.

OUTPUT

```
Training predictions:
Customer 1: target=1, output=1.000, predicted=1
Customer 2: target=1, output=1.000, predicted=1
Customer 3: target=1, output=0.992, predicted=1
Customer 4: target=1, output=1.000, predicted=1
Customer 5: target=1, output=0.999, predicted=1
Customer 6: target=0, output=0.000, predicted=0
Customer 7: target=0, output=0.001, predicted=0
Customer 8: target=0, output=0.010, predicted=0
Customer 9: target=0, output=0.007, predicted=0
Customer 10: target=1, output=0.979, predicted=1
Customer 11: target=1, output=1.000, predicted=1
Customer 12: target=0, output=0.018, predicted=0
Training accuracy: 100.0%
Test sample (frequent visits, responds to promotion, no previous purchase): output=0.996, predicted class=1
```

RESULT

The trained neural network predicts class 1 (Purchase) for the supplied customer sample.

Problem 4: Backpropagation Neural Network — Iris Binary Classification

AIM

To demonstrate a backpropagation neural network for binary classification using selected, normalized Iris-style flower measurements.

PROCEDURE

1. Prepare a small labelled dataset of normalized flower measurements.
2. Use two classes: 0 for one class of flower and 1 for the other class.
3. Initialize weights for the input, hidden, and output layers.
4. Perform forward propagation using sigmoid activation at the hidden and output layers.
5. Calculate the error and adjust weights using the backpropagation learning rule.
6. Test the model with a new flower measurement and display its predicted class.

PYTHON CODING

Run the complete code below using Python 3.

```
import math
import random

def sigmoid(value):
    # Clamp values to avoid overflow in exp().
    value = max(-60.0, min(60.0, value))
    return 1.0 / (1.0 + math.exp(-value))

def train_network(inputs, targets, hidden_count=4, epochs=5000, rate=0.5, seed=7):
    rng = random.Random(seed)
    input_count = len(inputs[0])
    # Small random weights and zero biases.
    w_hidden = [[rng.uniform(-0.5, 0.5) for _ in range(input_count)]
                for _ in range(hidden_count)]
    b_hidden = [0.0] * hidden_count
    w_output = [rng.uniform(-0.5, 0.5) for _ in range(hidden_count)]
    b_output = 0.0

    for _ in range(epochs):
        for x, target in zip(inputs, targets):
            # Forward propagation: input -> hidden layer.
            hidden = [
                sigmoid(sum(w * value for w, value in zip(weights, x)) + bias)
                for weights, bias in zip(w_hidden, b_hidden)
            ]
            # Hidden layer -> single sigmoid output.
            output = sigmoid(sum(w * value for w, value in zip(w_output, hidden))
                             + b_output)

            # Backpropagation: output-layer error signal.
            delta_output = (target - output) * output * (1.0 - output)
            old_w_output = w_output[:]
            for j in range(hidden_count):
                w_output[j] += rate * delta_output * hidden[j]
            b_output += rate * delta_output

            # Hidden-layer error signals use the old output weights.
            for j in range(hidden_count):
                delta_hidden = (delta_output * old_w_output[j]
                                * hidden[j] * (1.0 - hidden[j]))
                for k in range(input_count):
                    w_hidden[j][k] += rate * delta_hidden * x[k]
                b_hidden[j] += rate * delta_hidden

    def predict(x):
        hidden = [
            sigmoid(sum(w * value for w, value in zip(weights, x)) + bias)
            for weights, bias in zip(w_hidden, b_hidden)
        ]
        return sigmoid(sum(w * value for w, value in zip(w_output, hidden))
                       + b_output)

    return predict

def show_results(inputs, targets, names, sample, sample_name, hidden=4,
                 epochs=5000, rate=0.5, seed=7):
    model = train_network(inputs, targets, hidden, epochs, rate, seed)
    correct = 0
    print("Training predictions:")
    for x, target, name in zip(inputs, targets, names):
        probability = model(x)
        predicted = 1 if probability >= 0.5 else 0
        correct += int(predicted == target)
        print(f"{name}: target={target}, output={probability:.3f}, predicted={predicted}")
    print(f"Training accuracy: {100.0 * correct / len(targets):.1f}%")
    probability = model(sample)
    predicted = 1 if probability >= 0.5 else 0
    print(f"Test sample ({sample_name}): output={probability:.3f}, predicted class={predicted}")

# Normalized illustrative measurements: sepal length, petal length, petal width.
# Class 0 represents setosa-like samples; class 1 represents versicolor-like samples.
inputs = [
    [0.14, 0.07, 0.04], [0.18, 0.08, 0.05], [0.12, 0.06, 0.04],
    [0.20, 0.09, 0.05], [0.16, 0.07, 0.06], [0.22, 0.10, 0.05],
    [0.48, 0.45, 0.35], [0.52, 0.50, 0.38], [0.45, 0.42, 0.32],
    [0.56, 0.53, 0.40], [0.50, 0.47, 0.36], [0.46, 0.44, 0.34]
]
targets = [0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1]
names = [f"Flower {i}" for i in range(1, len(inputs) + 1)]
```

```
sample = [0.51, 0.48, 0.37]
sample_name = "normalized flower measurements"
show_results(inputs, targets, names, sample, sample_name,
             hidden=4, epochs=6000, rate=0.4, seed=17)
```

INPUT

The training inputs, target labels, and new test sample are defined directly in the program.

OUTPUT

```
Training predictions:
Flower 1: target=0, output=0.004, predicted=0
Flower 2: target=0, output=0.006, predicted=0
Flower 3: target=0, output=0.003, predicted=0
Flower 4: target=0, output=0.008, predicted=0
Flower 5: target=0, output=0.005, predicted=0
Flower 6: target=0, output=0.011, predicted=0
Flower 7: target=1, output=0.994, predicted=1
Flower 8: target=1, output=0.997, predicted=1
Flower 9: target=1, output=0.988, predicted=1
Flower 10: target=1, output=0.998, predicted=1
Flower 11: target=1, output=0.996, predicted=1
Flower 12: target=1, output=0.992, predicted=1
Training accuracy: 100.0%
Test sample (normalized flower measurements): output=0.997, predicted class=1
```

RESULT

The trained neural network predicts class 1 for the supplied normalized flower sample.

Problem 5: Backpropagation Neural Network — Health Risk Classification

AIM

To build a simple educational neural network using backpropagation to classify illustrative health-risk records from normalized measurements.

PROCEDURE

1. Create a small illustrative dataset with normalized age, activity level, and glucose-related measurement.
2. Label each record 1 for the example higher-risk class and 0 for the example lower-risk class.
3. Initialize weights and biases for the neural network.
4. Use forward propagation to calculate the output probability.
5. Apply backpropagation to calculate gradients and update weights during training.
6. Predict the class for a new sample and display the output.

PYTHON CODING

Run the complete code below using Python 3.

```
import math
import random

def sigmoid(value):
    # Clamp values to avoid overflow in exp().
    value = max(-60.0, min(60.0, value))
    return 1.0 / (1.0 + math.exp(-value))

def train_network(inputs, targets, hidden_count=4, epochs=5000, rate=0.5, seed=7):
    rng = random.Random(seed)
    input_count = len(inputs[0])
    # Small random weights and zero biases.
    w_hidden = [[rng.uniform(-0.5, 0.5) for _ in range(input_count)]
                for _ in range(hidden_count)]
    b_hidden = [0.0] * hidden_count
    w_output = [rng.uniform(-0.5, 0.5) for _ in range(hidden_count)]
    b_output = 0.0

    for _ in range(epochs):
        for x, target in zip(inputs, targets):
            # Forward propagation: input -> hidden layer.
            hidden = [
                sigmoid(sum(w * value for w, value in zip(weights, x)) + bias)
                for weights, bias in zip(w_hidden, b_hidden)
            ]
            # Hidden layer -> single sigmoid output.
            output = sigmoid(sum(w * value for w, value in zip(w_output, hidden))
                             + b_output)

            # Backpropagation: output-layer error signal.
            delta_output = (target - output) * output * (1.0 - output)
            old_w_output = w_output[:]
            for j in range(hidden_count):
                w_output[j] += rate * delta_output * hidden[j]
            b_output += rate * delta_output

            # Hidden-layer error signals use the old output weights.
            for j in range(hidden_count):
                delta_hidden = (delta_output * old_w_output[j]
                                * hidden[j] * (1.0 - hidden[j]))
                for k in range(input_count):
                    w_hidden[j][k] += rate * delta_hidden * x[k]
                b_hidden[j] += rate * delta_hidden

    def predict(x):
        hidden = [
            sigmoid(sum(w * value for w, value in zip(weights, x)) + bias)
            for weights, bias in zip(w_hidden, b_hidden)
        ]
        return sigmoid(sum(w * value for w, value in zip(w_output, hidden))
                       + b_output)

    return predict

def show_results(inputs, targets, names, sample, sample_name, hidden=4,
                 epochs=5000, rate=0.5, seed=7):
    model = train_network(inputs, targets, hidden, epochs, rate, seed)
    correct = 0
    print("Training predictions:")
    for x, target, name in zip(inputs, targets, names):
        probability = model(x)
        predicted = 1 if probability >= 0.5 else 0
        correct += int(predicted == target)
        print(f"{name}: target={target}, output={probability:.3f}, predicted={predicted}")
    print(f"Training accuracy: {100.0 * correct / len(targets):.1f}%")
    probability = model(sample)
    predicted = 1 if probability >= 0.5 else 0
    print(f"Test sample ({sample_name}): output={probability:.3f}, predicted class={predicted}")

# Illustrative educational data only; not medical data or a diagnostic tool.
# Features: normalized age, activity level (higher means more active), glucose indicator.
inputs = [
    [0.25, 0.85, 0.20], [0.35, 0.80, 0.25], [0.30, 0.70, 0.30],
    [0.45, 0.75, 0.35], [0.55, 0.65, 0.45], [0.65, 0.30, 0.75],
    [0.75, 0.25, 0.85], [0.80, 0.20, 0.90], [0.70, 0.35, 0.80],
    [0.60, 0.40, 0.70], [0.50, 0.60, 0.40], [0.85, 0.15, 0.95]
]
targets = [0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 1]
names = [f"Record {i}" for i in range(1, len(inputs) + 1)]
```

```
sample = [0.72, 0.25, 0.82]
sample_name = "illustrative normalized measurement"
show_results(inputs, targets, names, sample, sample_name,
             hidden=5, epochs=6000, rate=0.4, seed=19)
```

INPUT

The training inputs, target labels, and new test sample are defined directly in the program.

OUTPUT

```
Training predictions:
Record 1: target=0, output=0.000, predicted=0
Record 2: target=0, output=0.000, predicted=0
Record 3: target=0, output=0.001, predicted=0
Record 4: target=0, output=0.001, predicted=0
Record 5: target=0, output=0.016, predicted=0
Record 6: target=1, output=0.997, predicted=1
Record 7: target=1, output=0.999, predicted=1
Record 8: target=1, output=1.000, predicted=1
Record 9: target=1, output=0.997, predicted=1
Record 10: target=1, output=0.979, predicted=1
Record 11: target=0, output=0.012, predicted=0
Record 12: target=1, output=1.000, predicted=1
Training accuracy: 100.0%
Test sample (illustrative normalized measurement): output=0.999, predicted class=1
```

RESULT

The trained network predicts the illustrative higher-risk class (1) for the supplied sample; this result is for learning only.